0973. K Closest Points to Origin

Category: Heap / Priority Queue | LeetCode: https://leetcode.com/problems/k-clos

Problem

- Return the k points closest to (0,0) by Euclidean distance, in any order.

Intuition

- Compare squared distances (no sqrt needed).
- Keep a max-heap of size k: if a closer point arrives, evict the farthest of the current k.
- Quickselect gives O(n) average as a follow-up.

Brute Force

- Sort all points by distance: O(n log n).

Optimal Approach

- Max-heap ordered by xx + yy.
- For each point: offer; if size > k, poll.
- Return the heap contents.

Worked Example

```
points [[3,3],[5,-1],[-2,4]], k = 2.

[3,3] d=18  heap {18}
[5,-1] d=26 heap {26,18}
[-2,4] d=20 heap {26,20,18} -> evict 26
answer [[3,3],[-2,4]]
```

Java Solution

```java
public class KClosestPointsToOrigin {
    public int[][] kClosest(int[][] points, int k) {
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> Long.compare(dist(b), dist(a)));
        for (int[] p : points) {
            pq.offer(p);
            if (pq.size() > k) pq.poll();
        }
        return pq.toArray(new int[0][]);
    }

    private static long dist(int[] p) { return (long) p[0] * p[0] + (long) p[1] * p[1]; }
}
```

Complexity

- Time: O(n log k). Space: O(k).

Pitfalls

- Squared distance can reach 2 10^8 for constraints of 10^4 - fits in int, but use long-safe comparisons in general.